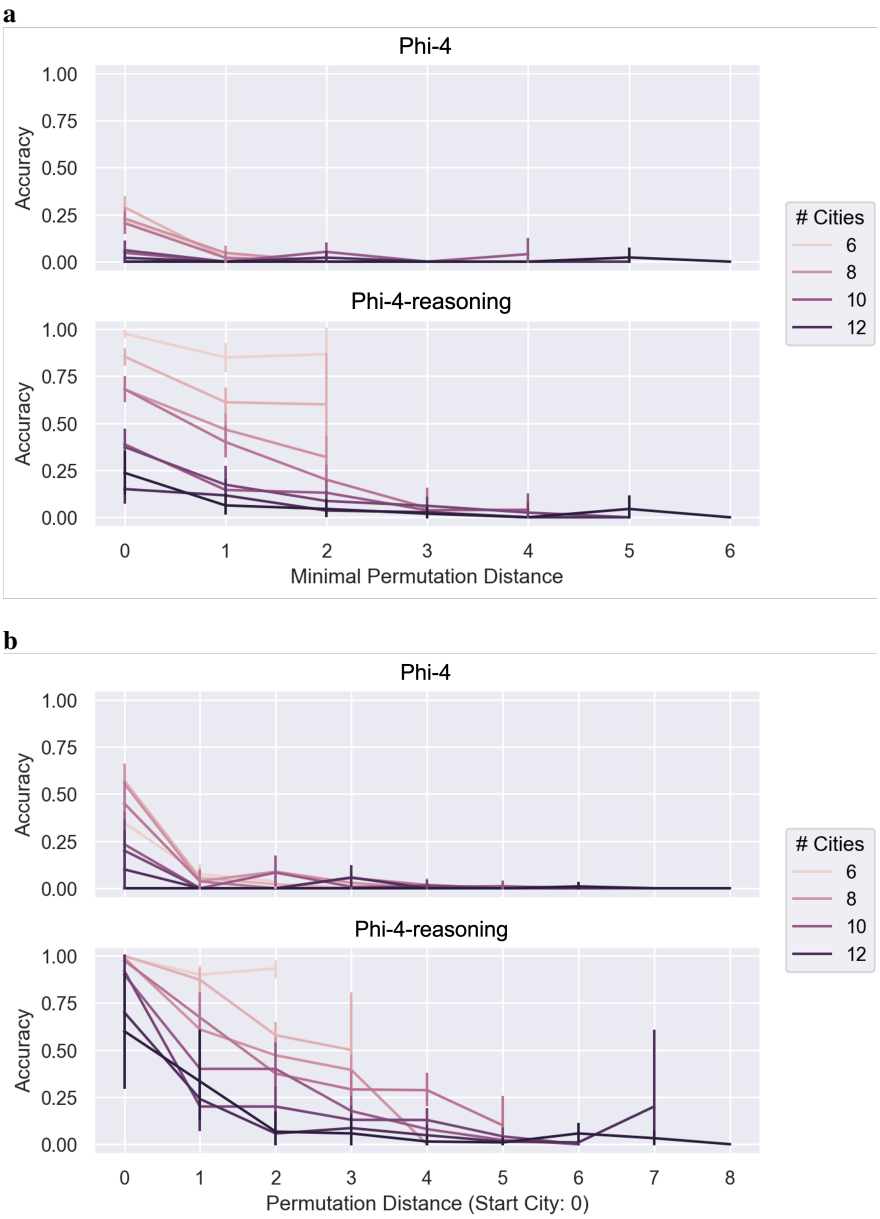

Figure 7: **a** The minimal NN-optimal edit distances with any city as the starting point. **b** The minimal NN-optimal edit distance using city 0 as the starting point. We observed a strong trend between edit distance and accuracy in **b**, even when controlling for the number of cities (and a somewhat weaker trend in **a**). This trend was especially strong for Phi-4-Reasoning, suggesting this model may effectively implement the nearest neighbor heuristic beginning at city 0.

Relatedly, `get_first_node` and `get_last_node` requires responding with the first or last node of a presented path (each node consists of numbers or capital letters and has between three and six elements). `get_predecessor` and `get_successor` receive a path and a node has input and need to return the predecessor or successor node respectively.

Example (`get_successor`)
Input: Graph: D-C-N-J, Node: C
Output: N

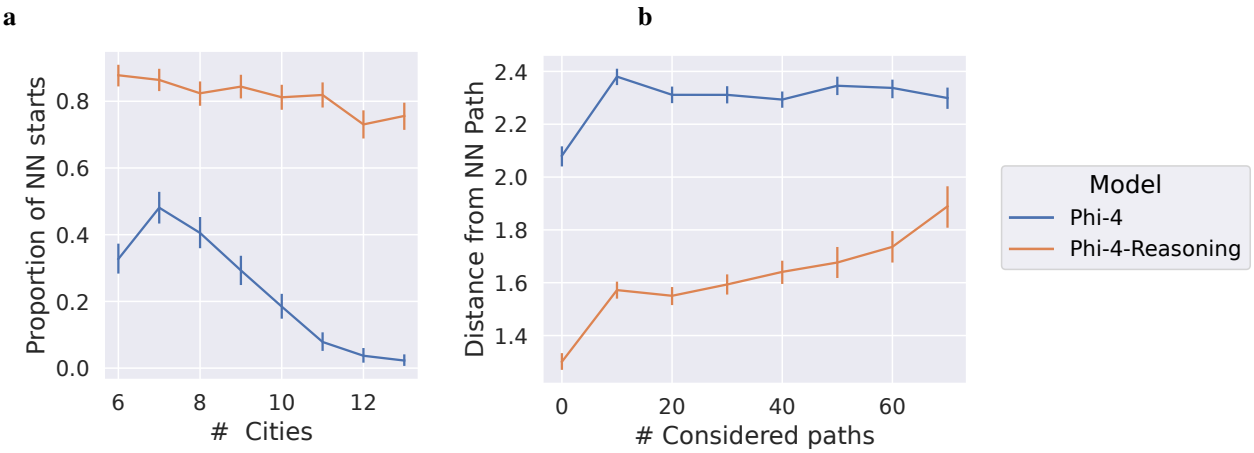

Figure 8: **a** We found that Phi-4 reasoning begins with the NN heuristic much more frequently than Phi-4-Base, especially when >8 cities are present. **b** Paths generated by Phi-4-Reasoning tend to have much lower distance on average from NN paths than those generated by Phi-4. Additionally, in Phi-4-Reasoning edit distance is lowest among the initial paths and increases as more candidate paths are selected.

E.2 Detailed methods and results

We extracted function vectors corresponding to each of these tasks using the same approach as Todd et al. (2024). We identified the 35 attention heads with the highest average indirect effect (see definition in Todd et al. (2024)) for Phi-4 and Phi-4-Reasoning and 20 attention heads the highest average indirect effect for Llama-3-8B. We injected these function vectors across all layers and analyzed the layer with the largest effect.

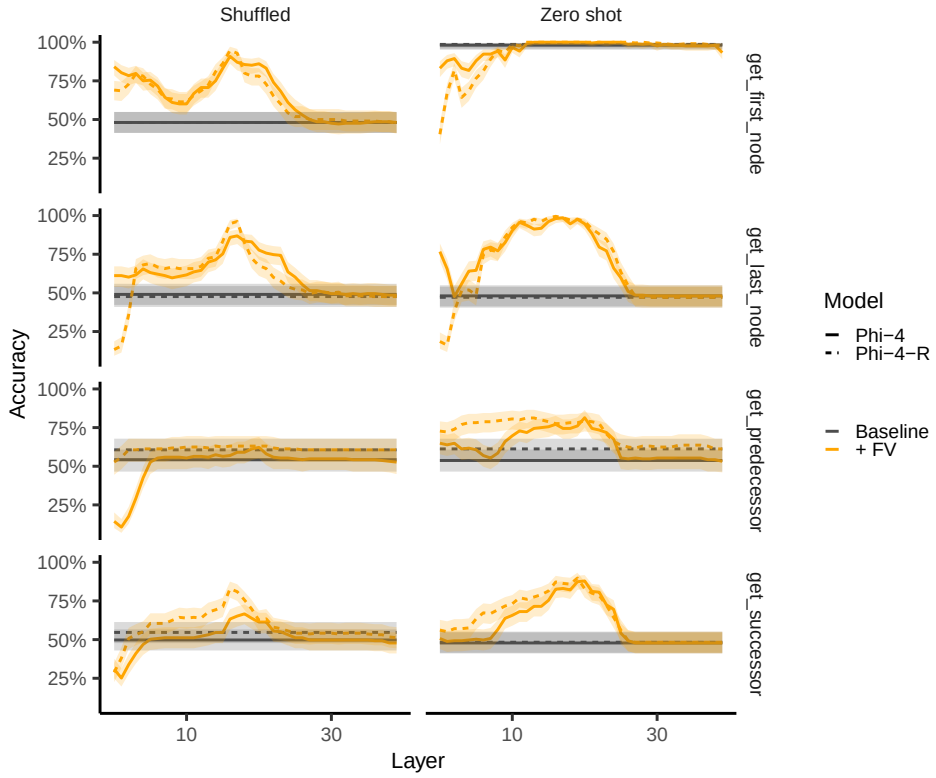

Figure 9: Accuracy on different graph operations after injecting the corresponding function vector into Phi-4 or Phi-4-Reasoning, either after shuffled in-context examples are in a zero-shot setting.

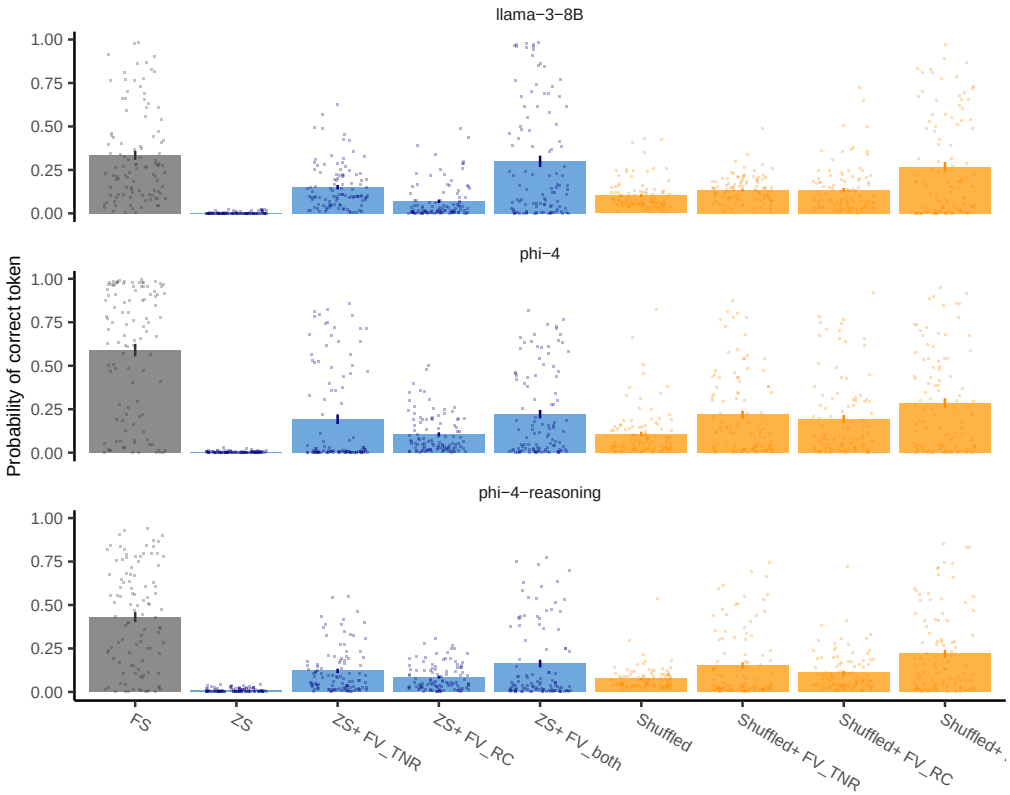

Figure 10: Average probability assigned to the correct tokens when injecting the Terminal_node_recognition and Reward_comparison FVs (FV_TNR and FV_RC) and the sum of both (FV_both). We compare the performance across Llama-3-8B, Phi-4, and Phi-4-Reasoning, and across a corrupted ICL and zero-shot context.

F Behavioral hallmarks on TSP

We consider three setups for injecting primitive vectors into TSP. In Figs. 3b,d, we only present the model with the TSP prompt and inject the primitive vectors throughout generation. We maximally generate 500 tokens. This is not sufficient to reliably generate a full answer, but is sufficient to test if the behavior is expressed. In Fig. 3c and Table 2, we add a sequence of randomly generated paths to the assistant response before injecting the function vectors. In both of these cases, we define several measures of different behavioral hallmarks. For these measures we automatically extract the set of paths generated in the model response using a regular expression. Below we specify our operational definitions of the different behavioral hallmarks:

1. % NN paths: What proportion of generated paths corresponds to a nearest-neighbor heuristic?
2. # Unique paths: How many valid TSP candidate paths are generated (after removing duplicate paths)?
3. Distance computation: What is the earliest token at which a distance computation occurs (operationalized as a sum of several numbers)? A lower number thus corresponds to a stronger expression of this behavior. If no distance computation occurs in the entire response, we set the value to 512.
4. Final answer: At what token does the model generate the `<final_answer>` token?
5. # Verifications: How many verification-related words are mentioned in the response?
6. # Comparison: How many comparison-related words are mentioned in the response?

Finally, when evaluating the impact of the `get_first_node` and `get_last_node` primitive vectors (Fig. 5c), we automatically extract a generated path from the middle of the reasoning trace (constraining our selection to only consider paths where the first and last node are not identical). We then generate the model response starting immediately after the generated path and injecting the corresponding primitive vector throughout. We generate 20 tokens and assess whether the first node that is mentioned corresponds to the first node of the path, the last node of the path or some other node.